

TP Integrador — Sistema de Atención al Cliente con IA

1. Descripción del caso de uso
2. Relevamiento de datos necesarios
3. Clasificación de los datos según su tipo
4. Modelo conceptual y lógico
5. Modelo de implementación según la tecnología elegida
6. Decisiones de normalización, embebido, referencia o desnormalización
7. Justificación de la tecnología seleccionada
8. Implementación mínima realizada
9. Datos de ejemplo utilizados
10. Consultas representativas
11. Propuesta para datos semiestructurados, no estructurados o vectoriales
12. Propuesta de arquitectura de datos
13. Estrategia de seguridad, permisos y aislamiento
14. Consideraciones de escalabilidad y rendimiento
15. Conclusiones

1. Descripción del caso de uso

- **Problema a resolver:** una empresa recibe consultas de sus clientes por múltiples canales (chat web, correo electrónico, WhatsApp, teléfono y formulario web) y hoy los atiende de manera dispersa: cada canal deja su propio registro, no existe un identificador único de caso que atraviese la interacción completa y el conocimiento generado al resolver una consulta se pierde una vez finalizada la atención.

Esto presenta las siguientes limitaciones:

1. No hay reutilización del conocimiento: consultas equivalentes se responden desde cero una y otra vez, porque el histórico no es consultable por contenido sino, en el mejor de los casos, por fecha o por cliente.
2. No hay medición confiable de la calidad del servicio: Sin un registro estructurado de los cambios de estado de cada caso, no es posible calcular tiempos de resolución, volumen por canal, carga por operador ni satisfacción del cliente.
3. No hay trazabilidad: no queda registrado quién intervino en cada momento de la atención.

La empresa desea incorporar asistencia basada en IA para **sugerir respuestas a partir de casos históricos ya resueltos, identificar las consultas más frecuentes y derivar a un operador humano los casos complejos**, conservando en todo momento el histórico de interacciones para mejorar la atención futura y auditar la calidad del servicio.

- **Usuarios principales:**

Usuario	Qué hace en el sistema	Qué necesita del modelo de datos
Cliente	Inicia una consulta por alguno de los canales habilitados, recibe una respuesta y califica la atención al cierre	Identidad validada (DNI, email o teléfono), acceso al historial de sus propios casos únicamente
Operador	Atiende los tickets asignados, revisa las respuestas sugeridas por la IA, las corrige o redacta las propias, cambia el estado del caso	Ver la conversación completa de sus casos, las sugerencias disponibles y su cola de trabajo
Supervisor	Controla la calidad de la atención, redistribuye carga entre operadores y audita casos puntuales	Indicadores agregados (tiempos de resolución, satisfacción por operador y por canal, tickets pendientes) y acceso de lectura al detalle de cualquier caso del equipo
Administrador	Gestiona altas y bajas de empleados, roles, canales habilitados y parámetros del sistema	Acceso a las tablas maestras y al esquema
Sistema de IA	Lee casos históricos resueltos para sugerir respuestas y detectar consultas frecuentes; escribe respuestas sugeridas	Un subconjunto de datos acotado, sólo de casos ya resueltos y validados , con el texto anonimizado y con las mismas restricciones de acceso que aplican al usuario que dispara la consulta

La distinción del sistema de IA como un usuario más (con sus propios permisos de lectura y escritura) es deliberada ya que si el modelo puede recuperar por similitud semántica un caso al que el operador no tendría acceso por la vía relacional habitual, el control de accesos queda vulnerado por una puerta lateral.

- **Procesos y funcionalidades:** la solución de datos debe soportar los siguientes procesos:
 1. **Identificación del cliente.** El cliente se autentica en la plataforma web o se identifica aportando datos privados (DNI, email o teléfono) al contactar por otro canal. Sus datos están validados al momento de generar el caso.

2. **Apertura de un ticket.** Todo contacto genera un ticket asociado obligatoriamente a un cliente, a un canal de origen tomado de un dominio predefinido y a un empleado responsable.
3. **Registro de la conversación.** Cada ticket agrupa una o más conversaciones, y cada conversación contiene una o más consultas concretas del cliente. Esta separación permite que un mismo caso incluya varias preguntas (“¿qué medios de pago aceptan?” y, a continuación, “¿puedo pagar en cuotas?”) sin abrir un ticket nuevo por cada una.
4. **Generación de respuestas.** Una consulta puede acumular varias respuestas: la sugerencia automática de la IA, la corrección del operador y la versión efectivamente enviada al cliente. El modelo debe distinguir el **origen** de cada respuesta (automática o humana) y marcar **cuál fue la que resolvió** la consulta, con la regla de que sólo puede existir una respuesta final por consulta.
5. **Derivación y cambio de estado.** El caso atraviesa un flujo de estados (abierto → en proceso → resuelto → cerrado, con posibilidad de reapertura) y puede pasar de un operador a otro. Cada transición queda registrada con fecha y empleado responsable, de modo que el historial sea reconstruible y no sobrescribible.
6. **Cierre y evaluación de satisfacción.** Al finalizar, el cliente puede calificar la atención en una escala acotada. La calificación es opcional.
7. **Cálculo de indicadores de servicio.** A partir del historial de estados y de las calificaciones, el sistema debe permitir calcular tiempos de resolución por canal, carga pendiente por operador, distribución de casos por canal y estado, y proporción de respuestas resueltas automáticamente frente a las resueltas por una persona.
8. **Identificación de consultas frecuentes y sugerencia de respuestas.** Sobre el corpus de consultas ya resueltas y sus respuestas validadas, el sistema debe permitir búsquedas por similitud semántica para agrupar temas recurrentes y proponer respuestas a casos nuevos.
9. **Control de accesos por rol y preservación del historial.** Cada rol accede únicamente al subconjunto de datos que le corresponde, y ninguna baja de cliente, empleado o ticket puede destruir el historial de auditoría asociado.

2. Relevamiento de datos necesarios

- **Información requerida:** el relevamiento se organizó por entidad del dominio.

Entidad	Datos a almacenar	Por qué se necesita
Cliente	Nombre, apellido, DNI, email, teléfono principal y alternativo, dirección, estado de actividad	Identificar y validar al cliente al abrir un caso y contactarlo por el canal correspondiente. El DNI y el email son identificadores únicos del cliente en el sistema
Empleado	Nombre, apellido, DNI, departamento, rol asignado, estado de actividad	Asignar responsables a los casos, trazar quién intervino en cada cambio de estado y calcular indicadores por operador
Rol	Descripción del rol (operador, supervisor, administrador)	Definir el nivel de acceso y las funciones habilitadas para cada empleado. (Se modela como entidad propia para poder incorporar roles nuevos sin modificar el esquema)
Ticket	Cliente asociado, empleado responsable actual, canal de origen, estado de actividad	Es la unidad de caso que atraviesa todos los canales y da un identificador único a la atención completa
Historial de estados (ticket log)	Ticket, empleado que ejecuta el cambio, fecha y hora, estado alcanzado	Reconstruir el ciclo de vida del caso, auditar derivaciones entre operadores y calcular tiempos de resolución
Conversación	Ticket al que pertenece, fecha de inicio, calificación de satisfacción	Agrupar el intercambio dentro de un caso y capturar la evaluación del cliente al cierre
Consulta	Conversación a la que pertenece, texto de la pregunta del cliente	Es el texto en lenguaje natural que dispara la atención y la unidad mínima sobre la que la IA busca casos similares

Entidad	Datos a almacenar	Por qué se necesita
Respuesta	Consulta a la que responde, texto de la respuesta, indicador de origen (humano o automático), indicador de respuesta final	Distinguir sugerencia de IA, corrección humana y respuesta efectivamente enviada
Índice semántico	Texto de la pregunta y de la respuesta final, canal de origen, vector de embedding, modelo utilizado, fecha de indexación, referencia a la consulta y respuesta de origen	Habilitar la búsqueda por similitud sin recalcular el embedding en cada consulta y sin bloquear las tablas operativas. Es un dato derivado, reconstruible a partir del núcleo transaccional

Riesgos asociados:

#	Riesgo	Descripción	Mitigación prevista en el diseño
R1	Exposición de datos personales	El modelo almacena DNI, email, teléfono y dirección de personas físicas. Una consulta mal filtrada expone información sensible	Control de accesos por rol nativo del motor y <i>Row Level Security</i> , de modo que el filtro no dependa de que la aplicación lo recuerde aplicar
R2	Fuga de datos sensibles por vía semántica	El cliente puede pegar su DNI, su email o datos de su tarjeta dentro del texto libre de la consulta. Ese texto se vectoriza e indexa, y luego puede reaparecer como “respuesta sugerida” ante un caso de otro cliente	Anonimizar/enmascarar el texto antes de generar el embedding
R3	Sugerencia de información incorrecta o desactualizada	Si una respuesta final se corrige después de haber sido indexada, el índice semántico sigue sugiriendo la versión anterior. El sistema propaga entonces una respuesta que la empresa ya invalidó	Reindexación asíncrona ante cualquier modificación del texto de una respuesta final; indexar únicamente respuestas marcadas como finales de casos ya resueltos, nunca sugerencias intermedias o descartadas
R4	Contaminación del corpus de conocimiento	La respuesta marcada como final es la que alimenta las sugerencias futuras	Restricción de unicidad de respuesta final por consulta, marca explícita de origen humano o automático y trazabilidad hacia el empleado responsable, para poder auditar
R5	Pérdida de trazabilidad por borrado físico	Eliminar un cliente, un empleado o un ticket destruiría el historial de auditoría asociado y falsearía retroactivamente los indicadores de servicio	Borrado lógico mediante el campo <code>activo</code> y políticas <code>ON DELETE RESTRICT</code> en todas las claves foráneas: ninguna baja puede arrastrar historial
R6	Inconsistencia de dominio en campos categóricos	Canales y estados cargados como texto libre derivan en valores equivalentes escritos de distinta forma (“WhatsApp”, “whatsapp”, “wsp”), que rompen silenciosamente toda agregación por canal o por estado	Dominios cerrados verificados por restricciones <code>CHECK</code> a nivel de motor

#	Riesgo	Descripción	Mitigación prevista en el diseño
R7	Transferencia de datos a un proveedor externo de embeddings	El texto de las consultas sale de la infraestructura de la empresa	Anonimización previa y, como alternativa evaluable, uso de un modelo de embeddings autoalojado si el volumen o la sensibilidad lo justifican
R8	Sesgo en los indicadores de desempeño	Los indicadores por operador (satisfacción promedio, carga pendiente) pueden usarse para evaluar personas sin considerar la complejidad diferencial de los casos ni el volumen de calificaciones recibidas	Exponer siempre el indicador junto a su denominador (cantidad de conversaciones calificadas) y documentar que se trata de una señal de proceso, no de una calificación individual concluyente

3. Clasificación de los datos según su tipo

- **Estructurados:** el núcleo completo del modelo relacional: `clientes`, `empleados`, `roles`, `tickets`, `ticket_logs`, `conversaciones`, `consultas` y `respuestas`. Las ocho tablas tienen esquema fijo, tipos de datos definidos, claves primarias y foráneas, y dominios cerrados controlados por restricciones `CHECK` (canal de origen, estado del ticket, calificación de satisfacción).
- **Semiestructurados:** ninguno en la implementación actual. No se incluyen columnas `JSON` / `JSONB`, `ARRAY` ni `hstore` porque, con el alcance actual del sistema, no se justifica su uso: todo el dominio se resuelve con esquema fijo. A futuro, si se suman flujos como trazas de generación de IA o metadatos variables por canal, ahí sí tendría sentido incorporar `JSONB` para esos casos puntuales.
- **No estructurados:** exactamente dos columnas de texto libre: `consultas.pregunta` y `respuestas.texto_respuesta`. Son el único dato del dominio en lenguaje natural sin estructura interna, y por eso son también los únicos candidatos a tratamiento vectorial.
- **Operacionales:** el registro día a día de la atención al cliente: apertura y seguimiento de `tickets`, su historial de estados en `ticket_logs`, y el contenido de `conversaciones`, `consultas` y `respuestas` de cada caso. Es el dato que sostiene la operación en tiempo real (qué caso está abierto, quién lo atiende, qué se respondió) y alimenta, en una segunda instancia, la capa analítica.
- **Analíticos:** los indicadores de calidad de servicio (tiempos de resolución, distribución de tickets por canal y estado, proporción de respuestas de IA vs. humanas, carga pendiente por operador). Hoy se calculan en caliente mediante las consultas agregadas del punto 10, directamente sobre las tablas operacionales. Se propone, como evolución (punto 12, etapa 2), materializarlos en un esquema `analitico` separado para no recalcular sobre el histórico completo en cada consulta de un tablero.
- **Sensibles:** los datos personales de `clientes` (`dni`, `email`, `telefono_1`, `telefono_2`, `direccion`); el texto libre de `consultas.pregunta` y `respuestas.texto_respuesta`, que puede contener datos personales que el cliente escribió sin que el sistema lo pida (riesgo R2); y `conversaciones.calificacion`, que combinada con el resto del caso puede usarse para evaluar indebidamente a un operador puntual (riesgo R8).
- **De auditoría o trazabilidad:** `ticket_logs`, diseñada como registro append-only (nunca admite `UPDATE` ni `DELETE`, ni siquiera para el rol administrador) que conserva cada cambio de estado con su `fecha` y el `empleado` responsable. A esto se suman los campos `respuestas.es_humano` y `respuestas.es_respuesta_final`, que trazan si cada respuesta fue generada por IA o por una persona y cuál fue la efectivamente enviada al cliente — la auditoría específica que pide el caso de uso sobre el desempeño del componente de IA.

4. Modelo conceptual y lógico

4.1 Modelo conceptual

4.1.1 Entidades principales

El dominio del caso de uso “Sistema de atención al cliente con IA” se modeló a partir de las siguientes entidades:

- **Cliente:** persona que inicia una consulta con la empresa a través de algún canal.
- **Ticket:** caso de atención abierto por un cliente. Es la entidad central que agrupa toda la interacción.
- **TicketLog:** historial de los estados por los que atraviesa un ticket a lo largo de su ciclo de vida, incluyendo qué empleado gestionó cada cambio. Resuelve tanto la trazabilidad del caso como la derivación entre empleados.
- **Empleado:** persona que trabaja en la empresa (operador, supervisor o administrador) y que puede gestionar tickets.
- **Rol:** perfil que determina qué puede hacer un empleado dentro del sistema.
- **Conversación:** intercambio dentro de un ticket. Un mismo ticket puede tener más de una conversación (por ejemplo, si el cliente retoma el caso días después).
- **Consulta:** pregunta puntual realizada por el cliente dentro de una conversación.
- **Respuesta:** respuesta asociada a una consulta, que puede haber sido generada por IA o por un empleado humano.

Se descartó modelar **Origen** como entidad propia: al tratarse de un conjunto acotado y estable de valores (chat , email , whatsapp , telefono , web) sin atributos propios, se representa como el atributo canal_origen de Ticket , evitando una tabla innecesaria.

4.1.2 Atributos relevantes por entidad

Entidad	Atributos
Cliente	id_cliente (PK), nombre, apellido, dni, email, telefono_1, telefono_2, direccion, activo
Ticket	id_ticket (PK), id_cliente (FK), id_empleado (FK, operador actual), canal_origen, activo
TicketLog	id_ticket_log (PK), id_ticket (FK), id_empleado (FK), estado, fecha
Empleado	id_empleado (PK), id_rol (FK), nombre, apellido, dni, departamento, activo
Rol	id_rol (PK), descripcion
Conversación	id_conversacion (PK), id_ticket (FK), calificacion, fecha
Consulta	id_consulta (PK), id_conversacion (FK), pregunta
Respuesta	id_respuesta (PK), id_consulta (FK), texto_respuesta, es_humano, es_respuesta_final

4.1.3 Relaciones y cardinalidades

Relación	Cardinalidad	Descripción
Cliente — Ticket	1 : N	Un cliente puede abrir varios tickets; cada ticket pertenece a un único cliente.
Ticket — TicketLog	1 : N	Un ticket acumula múltiples entradas de historial a lo largo de su ciclo de vida.
TicketLog — Empleado	N : 1	Cada entrada del historial fue gestionada por un único empleado; un empleado gestiona muchas entradas a lo largo del tiempo.

Relación	Cardinalidad	Descripción
Ticket — Empleado (operado por)	N : 1	Cada ticket tiene un único empleado asignado actualmente; un empleado puede tener asignados varios tickets.
Empleado — Rol	N : 1	Cada empleado tiene un único rol asignado; un rol puede corresponder a muchos empleados.
Ticket — Conversación	1 : N	Un ticket puede contener una o más conversaciones.
Conversación — Consulta	1 : N	Dentro de una conversación pueden registrarse varias consultas puntuales del cliente.
Consulta — Respuesta	1 : N	Una consulta puede tener más de una respuesta asociada (por ejemplo, la sugerencia generada por IA y la respuesta final corregida por un operador), preservando así la trazabilidad de ambas.

4.1.4 Justificación de decisiones de diseño

- **Ticket guarda el operador actual además de TicketLog:** es redundante (el dato ya se puede derivar del último registro de TicketLog), pero se acepta esa redundancia porque la consulta más frecuente del sistema es “qué tickets tiene asignados un operador ahora”, y resolverla sin recalcular sobre el historial es más rápido. TicketLog sigue siendo necesario aparte, porque es lo único que conserva el historial completo (quién tuvo el ticket antes, cuándo cambió de estado).
- **Consulta guarda el texto real que escribió el cliente, no una pregunta de un catálogo fijo:** cada fila de `Consulta` es la pregunta puntual de ese cliente en esa conversación (`pregunta`), variable y en lenguaje libre — no una entidad de “preguntas frecuentes” predefinidas contra la que se compara. Detectar que varias consultas distintas tratan el mismo tema (para identificar consultas frecuentes) se resuelve aparte, con búsqueda semántica, sin mezclarlo con el registro operacional.
- **Relación 1:N entre Consulta y Respuesta:** se descartó una relación 1:1 porque el sistema debe conservar tanto la respuesta sugerida automáticamente por la IA como la respuesta final enviada por un operador humano, cuando difieren. El atributo `es_respuesta_final` permite identificar cuál de las respuestas asociadas a una consulta fue la efectivamente enviada al cliente, y `es_humano` distingue el origen (IA o humano) de cada una.
- **Calificación asociada a la conversación, no al ticket:** dado que un ticket puede contener varias conversaciones, la evaluación de satisfacción (`calificacion`) se registró como atributo de `Conversación`, evitando la pérdida de información que ocurriría si se promediara o sobrescribiera a nivel de ticket.

4.1.5 Restricciones del dominio

- La calificación (`calificacion`) de una `Conversación` es otorgada por el cliente y debe estar en el rango de 1 a 5.
- En `Respuesta`, el campo `es_respuesta_final` indica cuál fue la respuesta que efectivamente resolvió la consulta o cerró la conversación. Por cada `Consulta`, solo una `Respuesta` puede tener `es_respuesta_final = true`, aunque existan varias respuestas asociadas.
- Todo `Ticket` debe abrirse con un `canal_origen` definido, tomado de un dominio cerrado de valores válidos.
- Todo `Ticket` debe estar asociado a un `Cliente`; no puede existir un ticket sin cliente asignado.
- El `Cliente` se autentica en la plataforma web o se identifica aportando datos privados (DNI, email o teléfono) al contactar por otro canal, por lo que sus datos están validados al momento de generar el `Ticket`.
- El `TicketLog` registra el historial de estados por los que atraviesa un `Ticket`. Cada entrada del log está asociada obligatoriamente a una fecha (`fecha`) y a un `Empleado` responsable de la gestión en ese momento.
- El campo `es_humano` de `Respuesta` es un dominio cerrado con dos valores posibles, implementado como `BOOLEAN` (`TRUE` = respuesta humana, `FALSE` = respuesta generada por IA).

4.2 Modelo lógico

4.2.1 Tabla roles

Columna	Restricción	Justificación
id_rol	PK	Identificador único del rol
descripcion	NOT NULL, UNIQUE	No deben existir roles duplicados con la misma descripción

4.2.2 Tabla clientes

Columna	Restricción	Justificación
id_cliente	PK	Identificador único del cliente
nombre	NOT NULL	Dato obligatorio para identificar al cliente
apellido	NOT NULL	Dato obligatorio para identificar al cliente
dni	UNIQUE, NOT NULL	No puede haber dos clientes con el mismo documento
email	UNIQUE, NOT NULL	Se usa como canal de contacto y posible identificador de acceso
telefono_1	NOT NULL	Canal de contacto principal obligatorio
telefono_2	NULL permitido	Contacto alternativo, opcional
direccion	NULL permitido	No es indispensable para dar de alta al cliente
activo	NOT NULL, DEFAULT true	Soft delete: permite dar de baja al cliente sin romper la integridad referencial con tickets

4.2.3 Tabla empleados

Columna	Restricción	Justificación
id_empleado	PK	Identificador único del empleado
id_rol	FK → roles(id_rol), NOT NULL, ON DELETE RESTRICT, ON UPDATE CASCADE	Todo empleado debe tener un rol asignado; no se permite borrar un rol con empleados activos
nombre	NOT NULL	Dato obligatorio
apellido	NOT NULL	Dato obligatorio
dni	UNIQUE, NOT NULL	No puede haber dos empleados con el mismo documento
departamento	NOT NULL	Necesario para trazabilidad organizacional
activo	NOT NULL, DEFAULT true	Soft delete: preserva la trazabilidad de tickets y logs asociados a un empleado dado de baja

4.2.4 Tabla tickets

Columna	Restricción	Justificación
id_ticket	PK	Identificador único del ticket

Columna	Restricción	Justificación
id_cliente	FK → clientes(id_cliente), NOT NULL, ON DELETE RESTRICT, ON UPDATE CASCADE	Todo ticket debe pertenecer a un cliente; no se permite eliminar un cliente con tickets asociados
id_empleado	FK → empleados(id_empleado), NOT NULL, ON DELETE RESTRICT, ON UPDATE CASCADE	Todo ticket debe tener un empleado asignado actualmente (operador actual)
canal_origen	NOT NULL, CHECK (IN ('chat', 'email', 'whatsapp', 'telefono', 'web'))	Restringe los valores a los canales soportados por el sistema
activo	NOT NULL, DEFAULT true	Soft delete: permite “eliminar” un ticket (ej. duplicado, error de carga) sin perder su historial de logs y conversaciones

4.2.5 Tabla **ticket_logs**

Columna	Restricción	Justificación
id_ticket_log	PK	Identificador único del registro de auditoría
id_ticket	FK → tickets(id_ticket), NOT NULL, ON DELETE RESTRICT, ON UPDATE CASCADE	Un log siempre pertenece a un ticket; el ticket no se borra físicamente, solo soft delete
id_empleado	FK → empleados(id_empleado), NOT NULL, ON DELETE RESTRICT, ON UPDATE CASCADE	Todo cambio de estado queda trazado a un empleado; no se permite borrar un empleado con logs asociados (preserva auditoría)
fecha	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Todo evento de log debe tener fecha de registro
estado	NOT NULL, CHECK (IN ('abierto', 'en_proceso', 'resuelto', 'cerrado', 'reabierto'))	Restringe los valores a los estados definidos del ciclo de vida del ticket

Tabla append-only: no admite **UPDATE** ni **DELETE** para ningún rol, incluido el administrador.

4.2.6 Tabla **conversaciones**

Columna	Restricción	Justificación
id_conversacion	PK	Identificador único de la conversación
id_ticket	FK → tickets(id_ticket), NOT NULL, ON DELETE RESTRICT, ON UPDATE CASCADE	Toda conversación pertenece a un ticket; el ticket nunca se borra físicamente, por lo que la conversación nunca queda huérfana
calificacion	NULL permitido, CHECK (BETWEEN 1 AND 5)	La calificación es opcional (se completa al cerrar la conversación) pero si existe debe estar en escala válida
fecha	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Toda conversación debe tener fecha de inicio

4.2.7 Tabla **consultas**

Columna	Restricción	Justificación
id_consulta	PK	Identificador único de la consulta
id_conversacion	FK → conversaciones(id_conversacion), NOT NULL, ON DELETE RESTRICT, ON UPDATE CASCADE	Toda consulta pertenece a una conversación; la conversación no tiene borrado propio, por lo que la consulta nunca queda huérfana

Columna	Restricción	Justificación
pregunta	NOT NULL, CHECK (LENGTH(pregunta) > 0)	El texto de la consulta no puede estar vacío

4.2.8 Tabla **respuestas**

Columna	Restricción	Justificación
id_respuesta	PK	Identificador único de la respuesta
id_consulta	FK → consultas(id_consulta), NOT NULL, ON DELETE RESTRICT, ON UPDATE CASCADE	Toda respuesta pertenece a una consulta; la consulta no tiene borrado propio, por lo que la respuesta nunca queda huérfana
texto_respuesta	NOT NULL, CHECK (LENGTH(texto_respuesta) > 0)	El texto de la respuesta no puede estar vacío
es_humano	NOT NULL, DEFAULT false	Debe quedar explícito si la respuesta la generó un agente humano o el sistema de IA
es_respuesta_final	NOT NULL, DEFAULT false	Debe quedar explícito si es la respuesta definitiva de la consulta o un intento intermedio

4.2.9 Resumen de políticas ON DELETE

Relación	Política	Justificación
empleados.id_rol → roles.id_rol	RESTRICT	No se elimina un rol en uso
tickets.id_cliente → clientes.id_cliente	RESTRICT	No se elimina un cliente con historial de tickets
tickets.id_empleado → empleados.id_empleado	RESTRICT	No se elimina un empleado con tickets asignados activos
ticket_logs.id_ticket → tickets.id_ticket	RESTRICT	El historial no se pierde; el ticket solo admite soft delete
ticket_logs.id_empleado → empleados.id_empleado	RESTRICT	Se preserva la auditoría aunque el empleado sea dado de baja
conversaciones.id_ticket → tickets.id_ticket	RESTRICT	El ticket nunca se borra físicamente
consultas.id_conversacion → conversaciones.id_conversacion	RESTRICT	La conversación nunca se borra físicamente
respuestas.id_consulta → consultas.id_consulta	RESTRICT	La consulta nunca se borra físicamente

4.2.10 Criterio de soft delete

No se realiza borrado físico sobre entidades con historial asociado. Solo **clientes**, **empleados** y **tickets** tienen flag **activo**; **roles**, **ticket_logs**, **conversaciones**, **consultas** y **respuestas** no lo necesitan por ser registros históricos inmutables o por depender del estado de su tabla padre.

4.2.11 Normalización

El modelo cumple 1FN, 2FN y 3FN en las 8 tablas: 1FN porque no hay campos multivaluados (**telefono_1** / **telefono_2** son columnas separadas) y toda PK es simple; 2FN se cumple automáticamente al no existir ninguna PK compuesta; 3FN se verificó revisando dependencias transitivas, sin encontrar ninguna (el caso límite analizado, **empleados.departamento** vs. **id_rol**, se descarta porque un mismo rol puede existir en más de un departamento).

Las únicas desnormalizaciones intencionales son `tickets.id_empleado` (si solo existiera `ticket_logs`, para saber el operador actual habría que buscar el último registro cada vez; y si solo existiera `tickets.id_empleado`, se perdería el historial de por quiénes pasó el ticket. Se mantienen las dos tablas: una para el dato actual, otra para la traza completa) y la futura `consultas_embeddings` (duplica el texto de la pregunta y la respuesta en vez de referenciarlo con un JOIN. Si tuviera que hacer ese JOIN contra `consultas/respuestas` en cada búsqueda por similitud, estaría leyendo las mismas tablas que en simultáneo usa el sistema para atender tickets en vivo, compitiendo por recursos con la operación real. Al duplicar el texto, la búsqueda semántica no toca esas tablas). En ambos casos se acepta que el dato duplicado pueda quedar momentáneamente desactualizado; el resto del núcleo mantiene consistencia fuerte en todo momento.

Diagrama disponible en `db/logico/logico_v.2.0.png`. Detalle completo en `db/logico/restricciones.md`.

5. Modelo de implementación según la tecnología elegida

- **Detalle del modelo:** el modelo se implementó como tablas relacionales en PostgreSQL: 8 tablas (`roles`, `clientes`, `empleados`, `tickets`, `ticket_logs`, `conversaciones`, `consultas`, `respuestas`) con relaciones 1:N resueltas mediante claves foráneas. No se necesitan tablas intermedias N:M porque el dominio no presenta relaciones muchos-a-muchos. No se eligió un modelo documental, columnar, de grafos ni vectorial como núcleo; la única extensión no relacional prevista es `pgvector`, para búsqueda semántica sobre consultas y respuestas, que aún no forma parte de la implementación mínima.
- **Estructuras específicas:**
 - **Claves primarias:** cada tabla usa una columna autonumérica (`IDENTITY`) como PK simple.
 - **Claves foráneas:** las 8 relaciones usan `FOREIGN KEY` con `ON DELETE RESTRICT` y `ON UPDATE CASCADE`, para no perder historial ante una baja.
 - **Restricciones de integridad:** `NOT NULL` en campos obligatorios, `UNIQUE` en identificadores naturales (`clientes.dni`, `clientes.email`, `empleados.dni`, `roles.descripcion`), `CHECK` en los dominios cerrados (`canal_origen`, `estado`, `calificacion` entre 1 y 5, longitud mínima de texto libre), y `DEFAULT` donde corresponde (`activo = true`, `fecha = CURRENT_TIMESTAMP`, `es_humano = false`, `es_respuesta_final = false`).
 - **Índices:** nueve índices sobre las columnas de los JOIN más frecuentes (uno por cada FK del camino cliente→ticket→conversación→consulta→respuesta), más un índice único parcial que garantiza una sola respuesta marcada como final por consulta.
 - **Vistas:** `vw_estado_actual_tickets`, que resuelve en una sola consulta el cliente, empleado asignado y último estado de cada ticket.

6. Decisiones de normalización, embebido, referencia o desnormalización

6.1 Estrategia general

El núcleo transaccional de la solución (`clientes`, `empleados`, `roles`, `tickets`, `ticket_logs`, `conversaciones`, `consultas`, `respuestas`) se diseñó como un modelo **relacional normalizado hasta 3FN** en PostgreSQL. Sobre ese núcleo se propone una extensión híbrida con **pgvector** para indexar semánticamente las consultas históricas y sus respuestas finales, con el objetivo de identificar consultas frecuentes y sugerir respuestas (detalle en `vectorial/modelo_vectorial.md`). Esta combinación permite aplicar, dentro del mismo motor de base de datos, los dos enfoques que pide la consigna: normalización clásica para el registro operativo del caso, y decisiones de embebido/referencia/desnormalización propias del mundo NoSQL para el componente de recuperación semántica.

6.2 Normalización del núcleo relacional

El detalle tabla por tabla está documentado en `db/logico/restricciones.md`. Que en resumen se detalla:

- **1FN — valores atómicos, sin grupos repetidos.** Se cumple en las 8 tablas: no existen columnas que almacenen listas (`clientes` separa `telefono_1` y `telefono_2` en vez de un campo concatenado), y toda fila se identifica por una PK simple autonumérica.
- **2FN — dependencia funcional completa de la clave.** Se cumple trivialmente porque ninguna tabla usa clave primaria compuesta: no hay relaciones muchos-a-muchos en el dominio (cada `ticket` tiene un único `cliente` y, en un momento dado, un único `empleado` asignado; cada `respuesta` pertenece a una única `consulta`), por lo que no existen tablas intermedias con dependencias parciales que resolver.
- **3FN — sin dependencias transitivas.** Se revisó cada tabla buscando atributos no clave que dependieran de otro atributo no clave. El caso límite analizado fue `empleados.departamento`: se descartó como dependencia transitiva de `id_rol` porque un mismo rol puede existir en más de un departamento, por lo que `departamento` es un atributo propio del empleado y no un dato derivable del rol.

Problemas que esta normalización evita, con ejemplos concretos del dominio:

Problema	Qué pasaría sin normalizar	Cómo lo evita el diseño
Redundancia	Repetir <code>nombre / apellido</code> del empleado en cada fila de <code>ticket_logs</code> (una por cada cambio de estado de cada ticket)	<code>ticket_logs.id_empleado</code> referencia a <code>empleados</code> ; el nombre se guarda una sola vez
Anomalía de inserción	No poder dar de alta un <code>rol</code> nuevo hasta que exista un empleado con ese rol, si el rol viviera como texto suelto dentro de <code>empleados</code>	<code>roles</code> es una entidad propia; se puede crear un rol sin empleados asignados todavía
Anomalía de actualización	Si la descripción de un rol se repitiera en cada empleado, cambiar el nombre de un rol obligaría a actualizar N filas, con riesgo de dejar alguna desactualizada	<code>empleados.id_rol</code> es FK a <code>roles</code> ; la descripción se actualiza en un único lugar
Anomalía de eliminación	Si <code>consultas / respuestas</code> fueran columnas embebidas en <code>tickets</code> , cerrar o depurar un ticket podría arrastrar la pérdida de toda la conversación	Cada entidad tiene su propio ciclo de vida y PK; además se usa <i>soft delete</i> (<code>activo = false</code>) en vez de <code>DELETE</code> físico
Inconsistencia de dominio	<code>canal_origen</code> como texto libre permitiría valores como “WhatsApp”/“whatsapp”/“wsp”	<code>CHECK (canal_origen IN (...))</code> fija un dominio cerrado de valores válidos

6.3 Desnormalización controlada

Se identificaron dos desnormalizaciones **intencionales**, documentadas y justificadas por patrón de consulta, no por descuido de diseño:

a) `tickets.id_empleado` (asignación actual) frente al histórico de `ticket_logs`. El empleado asignado actualmente a un ticket se refleja tanto en `tickets.id_empleado` como, implícitamente, en la última fila de `ticket_logs` de ese ticket. - *Ventaja*: la consulta más frecuente del sistema —“¿qué tickets tiene asignados cada operador ahora mismo?”— se resuelve con un filtro directo sobre `tickets`, sin calcular por cada ticket cuál es su log más reciente. - *Compromiso*: ambos valores deben mantenerse sincronizados (toda reasignación debe escribirse en ambas tablas, idealmente en la misma transacción). Se acepta este costo de escritura porque un ticket se reasigna pocas veces pero se consulta su estado actual constantemente.

b) Índice vectorial de consultas/respuestas frecuentes (extensión `pgvector`). Para identificar consultas frecuentes y sugerir respuestas, se decidió mantener una tabla desnormalizada de solo lectura (`consultas_embeddings`) que **referencia** la fila de origen (`id_consulta / id_respuesta`, para trazabilidad) pero **duplica (embebe)** una copia normalizada del texto de la pregunta y de la respuesta final junto a su vector, en lugar de resolver esto con un `JOIN` en caliente contra `consultas / respuestas`. - *Ventaja*: las búsquedas de similitud — potencialmente muy frecuentes si se ejecutan por cada consulta entrante— no compiten por bloqueos ni I/O con las tablas operativas, y se benefician de un índice `HNSW` dedicado. - *Compromiso*: si el texto original se corrige después

de indexado, el contenido embebido queda desactualizado hasta la próxima reindexación (proceso batch o disparado por evento). Se acepta **consistencia eventual** en este componente puntual, mientras el núcleo transaccional (estado del ticket, quién respondió, calificación) mantiene consistencia fuerte en todo momento. Esta es la misma lógica de “desnormalización controlada” propia de NoSQL, aplicada dentro del motor relacional gracias a pgvector.

6.4 Cómo la estructura responde a los patrones de consulta esperados

- *Selección y filtrado operativo* (tickets abiertos de un cliente, por canal o por operador) → índices sobre `tickets.id_cliente`, `tickets.id_empleado`, `tickets.canal_origen`.
- *Reconstrucción de una conversación completa* → cadena de JOINS `clientes` → `tickets` → `conversaciones` → `consultas` → `respuestas`, con integridad referencial garantizada.
- *Indicadores de calidad de servicio* (tiempos de resolución, satisfacción promedio por canal/operador) → agregaciones (`GROUP BY`, `AVG`) sobre `ticket_logs.fecha` y `conversaciones.calificacion`.
- *Auditoría de respuestas automáticas vs. humanas* → `respuestas.es_humano` y `respuestas.es_respuesta_final`, con la regla de unicidad de respuesta final por consulta.
- *Identificación de consultas frecuentes / sugerencia de respuestas* → `ORDER BY embedding <-> query_vector LIMIT N` sobre `consultas_embeddings`, combinable con filtros SQL convencionales en la misma sentencia.

7. Justificación de la tecnología seleccionada

7.1 Tecnología elegida

PostgreSQL como motor único, con la **extensión pgvector** habilitada sobre el mismo motor para el componente de búsqueda semántica. No se incorpora ningún motor NoSQL adicional (documental, clave-valor, columnar o de grafos) como parte del sistema de registro.

7.2 Justificación según los criterios de la consigna

- **Tipo de datos a almacenar.** El dominio combina datos estructurados con reglas de negocio estrictas (clientes, empleados, tickets, estados) con una porción de texto libre en lenguaje natural (`consultas.pregunta`, `respuestas.texto_respuesta`). Esta última porción es la única candidata a tratamiento vectorial (ver punto 11); el resto del dominio es naturalmente tabular.
- **Estructura y variabilidad de los datos.** El esquema es estable, los canales de atención, los estados de ticket y los roles son un conjunto cerrado y conocido de antemano (reforzado con `CHECK`), no datos que cambien de forma en cada registro. Esto favorece un esquema fijo y tipado por sobre un esquema flexible tipo documento.
- **Volumen esperado.** Es un sistema de atención al cliente pensado para empresas con un volumen de consultas bajo a mediano, no una plataforma de escala masiva (redes sociales, IoT). El volumen de tickets/consultas/respuestas es alto pero manejable por PostgreSQL.
- **Patrones de consulta.** Conviven tres patrones: transaccional (OLTP) sobre un ticket puntual, analítico/agregado para reportes de calidad de servicio (JOINS, `GROUP BY`, funciones de ventana), y búsqueda por similitud para sugerencia de respuestas. PostgreSQL + pgvector resuelve los tres dentro de un mismo motor y una misma transacción, evitando construir y sincronizar sistemas separados.
- **Relaciones entre entidades.** Son numerosas, jerárquicas y bien definidas (`cliente` → `ticket` → `conversación` → `consulta` → `respuesta`, `empleado` → `rol`), con reglas de integridad estrictas (una respuesta no puede existir sin una consulta; una consulta sólo puede tener una respuesta marcada como final). Este tipo de reglas se expresa de forma nativa con `FOREIGN KEY`, `CHECK` y transacciones; en un modelo documental habría que reimplementarlas a nivel aplicación, con mayor riesgo de inconsistencia.

- **Consistencia requerida.** El estado de un ticket, quién lo atendió y la calificación de satisfacción alimentan reportes de gestión: requieren consistencia fuerte y transacciones ACID. El único componente donde se acepta consistencia eventual es el índice vectorial de sugerencias, que no es fuente de verdad sino una ayuda de recuperación.
- **Seguridad y control de acceso.** El caso exige aislar lo que puede ver cada rol (cliente, operador, supervisor, administrador). PostgreSQL permite resolver esto con **roles nativos + Row Level Security (RLS)**, aplicando el control a nivel de motor en lugar de depender exclusivamente de que la aplicación recuerde aplicar el filtro correcto.
- **Necesidades de escalabilidad.** Moderadas en el estado actual, con margen de crecimiento vía escalabilidad vertical inmediata y, si es necesario, horizontal (réplicas de lectura, particionamiento). No se justifica, a esta escala, la complejidad de un sistema distribuido tipo Cassandra.
- **Complejidad operativa.** Un único motor para lo transaccional, lo analítico básico y lo vectorial reduce la cantidad de piezas de infraestructura a desplegar, respaldar, monitorear y asegurar, frente a combinar PostgreSQL + MongoDB + una base vectorial dedicada. Se optó por la combinación mínima que cubre los requerimientos reales del caso.

Ventajas y limitaciones frente a otras alternativas:

Alternativa	Por qué no se adoptó como base para este caso
MongoDB (documental)	El dominio tiene relaciones estables y reglas de integridad estrictas. Modelarlo como documentos obligaría a duplicar datos de cliente/empleado en cada documento relacionado, reproduciendo los problemas de redundancia y actualización que la normalización busca evitar.
Cassandra (columnar)	Pensada para volúmenes de escritura muy altos con consultas conocidas de antemano por clave de partición (streams de eventos/sensores). Este caso no tiene ese perfil de escritura y prioriza consistencia fuerte sobre disponibilidad extrema, lo opuesto al compromiso CAP típico de Cassandra.
Neo4j (grafos)	Las relaciones del dominio tienen profundidad conocida y acotada (pocos saltos de FK), no recorridos de longitud variable ni preguntas tipo “camino más corto”. Un grafo agregaría complejidad de modelado sin resolver mejor los patrones de acceso reales.
Redis (clave-valor)	Útil como complemento futuro (caché de respuestas frecuentes, estado temporal de una conversación en curso, <i>rate limiting</i>) pero no como sistema de registro: no ofrece integridad referencial ni consultas relacionales. Se deja como posible mejora de escalabilidad, no como parte del núcleo.
Base vectorial dedicada (Pinecone, Weaviate, Milvus, etc.)	Sumaría un segundo sistema a operar, sincronizar y asegurar por separado, con riesgo de que los metadatos de permisos queden desacoplados de la base transaccional. Con el volumen esperado, pgvector dentro de PostgreSQL cubre la necesidad sin ese costo operativo y sin duplicar el modelo de permisos.
PostgreSQL + pgvector (adoptada)	Cubre en un solo motor, integridad referencial y transacciones ACID para el núcleo operativo, consultas analíticas complejas para reportes de calidad de servicio, y búsqueda por similitud para consultas frecuentes/respuestas sugeridas — con un único modelo de seguridad (roles + RLS) aplicado de forma consistente a todos los datos, incluidos los vectores.

7.3 Límites reconocidos de la elección

Se reconoce que, si el volumen de búsquedas semánticas creciera órdenes de magnitud (por ejemplo, si la solución se ofreciera como plataforma multi-tenant a muchas empresas), podría justificarse migrar el componente vectorial a un motor dedicado. A la escala del caso propuesto (una empresa gestionando su propia atención al cliente), esa complejidad adicional no se justifica hoy.

8. Implementación mínima realizada

8.1 Alcance de la implementación

La implementación mínima se realizó en **PostgreSQL 16** sobre una base de datos denominada `bdia`. Se construyó el núcleo relacional completo definido en el modelo lógico, compuesto por las tablas `roles`, `clientes`, `empleados`, `tickets`, `ticket_logs`, `conversaciones`, `consultas` y `respuestas`.

La estructura física incluye:

- Identificadores autonuméricos mediante columnas `IDENTITY`;
- Claves primarias y foráneas para garantizar la integridad referencial;
- Restricciones `NOT NULL`, `UNIQUE` y `CHECK` para controlar los dominios;
- Políticas `ON DELETE RESTRICT` y `ON UPDATE CASCADE` en las relaciones;
- Borrado lógico mediante el campo `activo` en clientes, empleados y tickets;
- Nueve índices para claves foráneas y patrones de consulta frecuentes;
- Un índice único parcial que garantiza una sola respuesta final por consulta;
- Una vista que presenta el cliente, empleado asignado y último estado de cada ticket.

La extensión `pgvector` y la tabla `consultas_embeddings` se mantienen como propuesta del componente vectorial descrito en el punto 11; no forman parte del núcleo transaccional mínimo implementado en esta etapa.

8.2 Scripts desarrollados

Los archivos deben ejecutarse respetando el siguiente orden:

1. `db/fisico/01_creacion_tablas.sql`: Crea las ocho tablas, sus tipos de datos, claves y restricciones de integridad.
2. `db/fisico/02_indices_vistas.sql`: Crea los índices de rendimiento, la restricción de respuesta final única y la vista `vw_estado_actual_tickets`.
3. `data/03_datos_ejemplo.sql`: Carga el conjunto sintético utilizado para comprobar las relaciones y consultas.
4. `db/fisico/04_validaciones.sql`: Ejecuta trece pruebas automáticas sobre las estructuras creadas.

Todos los scripts utilizan transacciones. Los tres primeros finalizan con `COMMIT` para evitar una implementación parcialmente cargada ante un error. El script de validaciones finaliza con `ROLLBACK`, ya que realiza inserciones y actualizaciones intencionalmente inválidas para comprobar las restricciones sin alterar los datos de ejemplo.

9. Datos de ejemplo utilizados

9.1 Estructura de prueba

Se generó un conjunto de datos completamente **sintético**, sin información personal real, con el siguiente volumen:

Tabla	Registros	Casos representados
<code>roles</code>	3	Operador, supervisor y administrador

Tabla	Registros	Casos representados
empleados	5	Operadores de distintas áreas, un supervisor y un administrador
clientes	6	Clientes con datos de contacto obligatorios y opcionales
tickets	10	Todos los canales admitidos y un caso de borrado lógico
ticket_logs	31	Estados abiertos, en proceso, resueltos, cerrados y reabiertos
conversaciones	10	Casos con y sin calificación de satisfacción
consultas	12	Conversaciones con una o varias preguntas
respuestas	17	Respuestas de IA, humanas, intermedias y finales

Los datos permiten representar tickets en distintas etapas del ciclo de vida, derivaciones entre operadores, diferentes canales de atención, conversaciones pendientes y finalizadas, calificaciones entre 1 y 5 y consultas con múltiples respuestas. Los identificadores se establecieron explícitamente para facilitar la lectura de las relaciones y, al finalizar la carga, se sincronizaron las secuencias autonuméricas para permitir nuevas inserciones.

9.2 Validación de coherencia

El archivo `db/fisico/04_validaciones.sql` comprueba automáticamente:

- La existencia de las ocho tablas y las cantidades esperadas de datos;
- La unicidad del DNI de los clientes;
- La obligatoriedad del correo electrónico;
- El rechazo de claves foráneas inexistentes;
- Los dominios cerrados de canales y calificaciones;
- El rechazo de consultas vacías;
- La regla de una sola respuesta final por consulta;
- La aplicación de valores predeterminados;
- La conservación del historial mediante borrado lógico;
- El cálculo correcto del último estado a través de la vista;
- La existencia del índice único parcial y de la vista implementada.

Cada prueba informa un mensaje `OK` cuando PostgreSQL rechaza o procesa el caso de la forma esperada. Como el script completo finaliza con `ROLLBACK`, los registros temporales utilizados en las pruebas no permanecen almacenados.

10. Consultas representativas

Breve explicación de la utilidad de cada una de las 5 consultas mínimas requeridas, adjuntando el código correspondiente. `db/consultas/05_consultas_representativas.sql` tiene 8 consultas numeradas 1 a 8; estas 5 son la 1, 3, 7, 8 y 4 de ese archivo, en ese orden, elegidas para cubrir las cinco categorías que pide la consigna: 1. **Consulta 1 (Selección y filtrado):** ¿Qué tickets tiene pendiente cierto operador? Muestra los trabajos que todavía tiene por terminar el operador.

```
SELECT id_ticket, cliente, canal_origen, estado_actual, fecha_ultimo_estado
FROM vw_estado_actual_tickets
WHERE id_empleado = 3 AND activo = TRUE AND estado_actual IN ('abierto', 'en_proceso', 'reabierto')
ORDER BY fecha_ultimo_estado;
```

2. **Consulta 2 (Información relacionada / JOINS):** Esta consulta responde a la pregunta: ¿Qué consultas realizó un cliente y qué respuestas fueron generadas por IA o enviadas por una persona?

```
SELECT t.id_ticket, CONCAT_WS(' ', c.nombre, c.apellido) AS cliente, conv.id_conversacion,
       q.id_consulta, q.pregunta, resp.id_respuesta, resp.texto_respuesta,
       CASE
         WHEN resp.es_humano = TRUE THEN 'Humana'
         ELSE 'IA'
       END AS origen_respuesta, resp.es_respuesta_final
FROM tickets AS t
INNER JOIN clientes AS c ON c.id_cliente = t.id_cliente
INNER JOIN conversaciones AS conv ON conv.id_ticket = t.id_ticket
INNER JOIN consultas AS q ON q.id_conversacion = conv.id_conversacion
LEFT JOIN respuestas AS resp ON resp.id_consulta = q.id_consulta
WHERE t.id_ticket = 9
ORDER BY q.id_consulta, resp.id_respuesta;
```

3. **Consulta 3 (Agregaciones / Indicadores):** Esta consulta responde el porcentaje de respuestas finales de IA y humanas.

```
SELECT
  CASE
    WHEN es_humano = TRUE THEN 'Humana'
    ELSE 'IA' END AS origen_respuesta,
    COUNT(*) AS cantidad_respuestas_finales,
    ROUND( 100.0 * COUNT(*) / SUM(COUNT(*)) OVER (), 2 ) AS porcentaje
FROM respuestas
WHERE es_respuesta_final = TRUE
GROUP BY es_humano
ORDER BY cantidad_respuestas_finales DESC;
```

4. **Consulta 4 (Toma de decisiones):** Esta consulta responde la carga de trabajo pendiente por cada operador. Para que el supervisor pueda ver cuanto trabajo tiene cada operador y asignar tareas en base a ello.

```
SELECT
  e.id_empleado,
  CONCAT_WS(' ', e.nombre, e.apellido) AS operador,
  COUNT(actual.id_ticket) AS tickets_pendientes
FROM empleados AS e
INNER JOIN roles AS r ON r.id_rol = e.id_rol
LEFT JOIN vw_estado_actual_tickets AS actual ON actual.id_empleado = e.id_empleado AND actual.activo
  = TRUE AND actual.estado_actual IN ('abierto', 'en_proceso', 'reabierto')
WHERE r.descripcion = 'Operador' AND e.activo = TRUE
GROUP BY e.id_empleado, e.nombre, e.apellido
ORDER BY tickets_pendientes DESC, operador;
```

5. **Consulta 5 (Optimización mediante Índices/Vistas):** La vista vw_estado_actual_tickets evita repetir en cada consulta los JOIN necesarios para relacionar tickets, clientes, empleados, roles y el último registro de ticket_logs. Esto simplifica las consultas operativas y centraliza la lógica utilizada para determinar el estado actual de cada ticket. Además, la búsqueda del último registro se optimiza mediante el índice compuesto idx_ticket_logs_ticket_fecha, que busca por ticket y el orden descendiente de la fecha.

```
SELECT canal_origen, estado_actual, COUNT(*) AS cantidad_tickets
FROM vw_estado_actual_tickets
WHERE activo = TRUE
GROUP BY canal_origen, estado_actual
ORDER BY canal_origen, cantidad_tickets DESC;
```

11. Propuesta para datos semiestructurados, no estructurados o vectoriales

El desarrollo completo de este punto está en `docs/09_datos_semiestructurados_no_estructurados_vectorial.md` (inventario dato por dato, comparación de alternativas de almacenamiento, metadatos del índice semántico, filtros por permisos y análisis de riesgos).

11.1 Inventario real del esquema implementado

De las 8 tablas y aproximadamente 40 columnas del modelo físico:

- **Estructurado:** todo el núcleo, con dominios cerrados por `CHECK` (`canal_origen`, `estado`, `calificacion`) e integridad por `FOREIGN KEY`.
- **Semiestructurado:** **ninguno**. No existe una sola columna `JSON` / `JSONB`, `ARRAY`, `hstore` ni `XML` en `01_creacion_tablas.sql`.
- **No estructurado:** exactamente **dos columnas**, `consultas.pregunta` y `respuestas.texto_respuesta`, ambas `TEXT` con `CHECK LENGTH > 0`.
- **De auditoría:** `ticket_logs` es un log de eventos de negocio, pero **de esquema fijo y dominio cerrado**: no es un log semiestructurado y no corresponde sacarlo del modelo relacional.
- **Relaciones:** jerarquía lineal de profundidad fija (`cliente` → `ticket` → `conversación` → `consulta` → `respuesta`); **ninguna relación muchos-a-muchos en las 8 tablas**. No hay estructura “altamente conectada” que justifique un grafo, en línea con lo argumentado en 7.2.

11.2 Manejo de datos complejos: JSON/JSONB sí, pero acotado a tres lugares

JSONB se justifica **solo donde la variabilidad de esquema es real**, no como estrategia general. Tres candidatos concretos, hoy inexistentes:

Dato	Representación	Motivo
Metadatos técnicos por canal (<code>message_id</code> de email, <code>wa_id</code> , duración de llamada, sesión de chat web)	<code>tickets.metadatos_canal</code> JSONB	Los atributos cambian según el valor de <code>canal_origen</code> ; columnas nullables por canal o un esquema EAV son peores y obligan a migrar el esquema al habilitar un canal nuevo
Trazas de generación de una sugerencia de IA (modelo, tokens, confianza, casos recuperados como contexto)	<code>respuestas.metadatos_generacion</code> JSONB	Cambia con cada versión del modelo; hace auditable la sugerencia (mitiga R4) sin un <code>ALTER TABLE</code> por proveedor. Hoy <code>es_humano</code> dice <i>que</i> fue automática, no <i>cómo</i>
Parámetros del sistema (umbral de similitud, cantidad de sugerencias, canales habilitados)	<code>parametros</code> (<code>clave</code> TEXT PK, <code>valor</code> JSONB)	Claves heterogéneas y volumen mínimo. La sección 1 atribuye “parámetros del sistema” al Administrador y no existe tabla que lo soporte

Regla adoptada: ningún dato que alimente un indicador de gestión, una FK o una restricción de negocio vive dentro de un JSONB. El JSONB documenta lo accesorio; el esquema tipado garantiza lo que se mide. Por contraste, el motivo de una derivación —que el enunciado pide y hoy queda implícito en `ticket_logs` (`ticket` 5: logs 14 y 15, mismo estado, distinto empleado)— **no** debe ir a JSONB: su estructura es estable y necesita FK a `empleados`, así que corresponden columnas nuevas (`id_empleado_anterior`, `motivo`).

No se incorpora ninguna colección NoSQL, confirmando 7.2. Los binarios que el caso implica pero el modelo no tiene (adjuntos de email, foto de producto dañado, audio del canal `telefono` — hoy el ticket 5 presupone una transcripción no modelada) corresponden a **almacenamiento de objetos externo + tabla de metadatos**, no a `BYTEA` ni a un motor documental.

11.3 Enfoque vectorial: se justifica, acotado a 2 columnas

Qué se vectorizaría: únicamente el par `consultas.pregunta` + su `respuestas.texto_respuesta` final, en casos **cerrados y vigentes** — 10 pares tienen respuesta final en el conjunto de ejemplo, pero el criterio corregido más abajo reduce el corpus realmente elegible a 2.

Por qué la búsqueda tradicional no alcanza: es un límite de expresividad, no de rendimiento. El FTS nativo (`tsvector` + GIN, config `spanish`, con `unaccent`) es exacto, barato y **debería implementarse igual, antes que los vectores**; pero no resuelve **sinonimia ni paráfrasis**, que es el problema real (“*clave*” y “*contraseña*” no comparten lexema). La prueba está en los datos del proyecto: la consulta 1 (“*Como puedo restablecer mi contrasena?*”) y la consulta 11 (“*No puedo ingresar a la aplicacion movil*”) son el mismo problema de fondo —la propia sugerencia de IA para la 11 (`id_respuesta` 15, no la respuesta final) ya lo detecta: “*actualizar la aplicacion y restablecer la contrasena*”— y sin embargo **las dos consultas no comparten una sola palabra de contenido** entre sí.

Metadatos que acompañan al vector. El diseño de `vectorial/modelo_vectorial.md` incorpora, además de la referencia a `id_consulta` / `id_respuesta`, las columnas que la RLS y la trazabilidad del componente necesitan. Las tres decisivas:

1. **`id_cliente` e `id_ticket`**. Sin ellos, la RLS que ese documento declara no sería implementable sin el JOIN a `consultas` → `conversaciones` → `tickets` que la desnormalización existe para evitar.
2. **`es_humano` de la respuesta final**. En los datos de ejemplo **3 de las 10 respuestas finales son puramente de IA** (`id_respuesta` 3, 12, 17): indexarlas sin marca haría que el sistema se alimente de su propia salida, que es R4 sin mitigación efectiva.
3. **`fecha_resolucion`** — necesaria porque no es derivable de forma barata desde el núcleo: **ni `consultas` ni `respuestas` tienen columna de fecha propia**. La recencia es el filtro más importante: una resolución vieja puede ser hoy incorrecta aunque siga siendo la más parecida.

Las otras seis, también incorporadas: `hash_contenido` y `vigente` para reindexar sin duplicar y retirar un caso sin `DELETE`; `UNIQUE (id_consulta)` para que la reindexación no duplique filas ni infle los conteos de “consultas frecuentes”; el filtro por canal como re-ranking blando en lugar de `WHERE canal_origen = ...`, que partiría el corpus en cinco por una razón irrelevante al contenido; el reemplazo del `GROUP BY contenido_pregunta` de la segunda consulta —que contaba duplicados literales y no similares— por conteo de vecinos por radio; el índice HNSW diferido, porque es aproximado y con un corpus chico rinde peor que el scan exacto; y el umbral de distancia obligatorio en toda consulta, porque sin él la búsqueda siempre devuelve *k* resultados aunque ninguno sirva.

Corrección del criterio de indexación, con evidencia en los datos de ejemplo. Indexar por `es_respuesta_final = TRUE` sin mirar el estado del ticket es insuficiente en dos sentidos verificables:

- **resuelto no es un estado terminal** en este modelo: existe `reabierto`, y el ticket 8 recorre `abierto` → `resuelto` → `reabierto` → `en_proceso`. Indexar al llegar a `resuelto` cargaría al corpus un caso que después se probó no resuelto.
- **Hay respuestas finales en tickets dados de baja:** el **ticket 7 tiene `activo = FALSE`** (borrado lógico por duplicado) y su consulta 7 **sí tiene respuesta final** (`id_respuesta` 10). Con el criterio actual, ese caso entraría al índice y sería sugerible: el riesgo de “documentos eliminados pero presentes en el índice” ya está materializado en el conjunto de prueba.

Riesgos y mitigación. Sobre los ya identificados en la sección 2, los específicos del componente semántico:

- **Desactualización silenciosa.** R3 cubre el caso en que el texto cambia. No cubre el caso en que **el texto no cambia y el mundo sí**: “*Se aceptan tarjetas de credito, debito y transferencia bancaria*” (respuesta 12) sigue idéntica cuando la empresa deja de aceptar transferencias, y **ningún trigger lo detecta porque no hubo `UPDATE`**. Solo lo mitigan la ventana de recencia y la revisión periódica. Riesgo abierto.
- **Falsos positivos plausibles.** Las consultas 9 (“*Que medios de pago aceptan?*”) y 10 (“*Puedo pagar la compra en cuotas?*”) son vecinas muy cercanas con respuestas **no intercambiables**. La negación agrava el problema: “*puedo cancelar*” y “*no puedo cancelar*” son casi idénticos como vectores y opuestos como intención.

- **Autorización.** `roles` tiene **3 filas: Operador, Supervisor, Administrador**, y modela roles de negocio de empleados, no identidades de motor. La identidad de base de datos para el “Sistema de IA” que la sección 1 describe como usuario con permisos propios vive en `db/fisico/05_seguridad_permisos.sql` (`rol_sistema_ia`, sección 13): solo puede insertar sugerencias (`es_humano = false`) y no tiene lectura propia amplia — la recuperación corre con el rol de quien pregunta, que es exactamente “las mismas restricciones de acceso que aplican al usuario que dispara la consulta” que pide la sección 1, y evita la “puerta lateral” que advierte. Lo que falta es acotado: `consultas_embeddings` no existe físicamente todavía — el diseño ya especifica columnas propias `id_cliente / id_ticket` para que, cuando se cree, se le pueda aplicar la misma RLS sin reintroducir el JOIN que la desnormalización busca evitar (`vectorial/modelo_vectorial.md`, sección 13).
- **Privacidad.** La anonimización debe ocurrir **antes de calcular el embedding**, como ya indica R2, porque el vector conserva información del texto original y existen ataques de inversión. Además queda una tensión abierta: `clientes.activo = FALSE` se documenta como “anonimización de datos personales”, pero **nada anonimiza el texto de sus consultas ya indexadas**.
- **Pérdida de contexto.** La consulta 12 (“*Cuanto tiempo dura el enlace para cambiar la contraseña?*”) solo se entiende leída después de la consulta 1 de su misma conversación; indexada suelta es un fragmento huérfano.
- **Volumen.** Con 10 pares indexables el componente es **demostrativo, no útil**: su valor aparece a partir de algunos cientos de casos cerrados.

11.4 Conclusión del punto

Sí se justifica una solución vectorial, acotada y condicionada. Se justifica porque el enunciado exige sugerir respuestas a partir de casos históricos y el obstáculo es la sinonimia, que ninguna técnica léxica resuelve sin un glosario mantenido a mano indefinidamente. Se acota a **2 columnas de las ~40 del esquema**: el resto del dominio requiere igualdad, rango o agregación, y para eso el B-tree es exacto, más rápido y auditable. Y se condiciona a tres requisitos sin los cuales hace más daño que bien: anonimización previa al cálculo del embedding; RLS efectiva sobre la tabla de vectores (lo que exige `id_cliente / id_ticket` y una identidad de base de datos para la IA); y la sugerencia entregada **al operador como borrador**, nunca automáticamente al cliente.

Esa tercera condición ya está soportada por el modelo relacional y es su mejor decisión de diseño para este punto: la sugerencia de IA vive como respuesta no final y un humano crea la final, patrón visible en las cadenas 1→2, 5→6 y 13→14 de los datos de ejemplo.

No se justifica, en cambio, una base vectorial dedicada (`pgvector` en el mismo motor cubre la necesidad con un único modelo de permisos, según 7.2), ni construir el componente antes que el FTS nativo, que hoy tampoco existe y resuelve el caso más frecuente a un costo mucho menor.

12. Propuesta de arquitectura de datos

El desarrollo completo está en `docs/10_arquitectura_de_datos.md` (capa por capa, ciclo de vida extremo a extremo, consistencia y reconstruibilidad por capa, seguridad, etapas de implementación y umbrales de evolución). Lo que sigue es la síntesis.

12.1 Punto de partida

De las nueve capas que la arquitectura contempla, **hoy existe una**: la operacional (8 tablas, 9 índices, 1 vista), con roles de PostgreSQL y RLS ya implementados y probados sobre ella (`db/fisico/05_seguridad_permisos.sql`, sección 13). No hay ingesta, ni datos crudos, ni capa de procesamiento, ni almacenamiento analítico como tal — los indicadores se calculan en caliente con las consultas de `db/consultas/` —, ni la tabla de embeddings, ni consumidores (no hay código de aplicación). Por eso el criterio rector es que **cada capa se pueda incorporar por separado y el sistema funcione sin las que faltan**.

Tres reglas gobiernan el diseño: una sola fuente de verdad (el núcleo relacional; todo lo demás es derivado y reconstruible); ninguna capa derivada es indispensable para atender un caso; y el dato personal se degrada al avanzar (identificable en el crudo, con RLS en el operacional, **anonimizado antes de entrar a la capa de IA**).

12.2 Diagrama de arquitectura

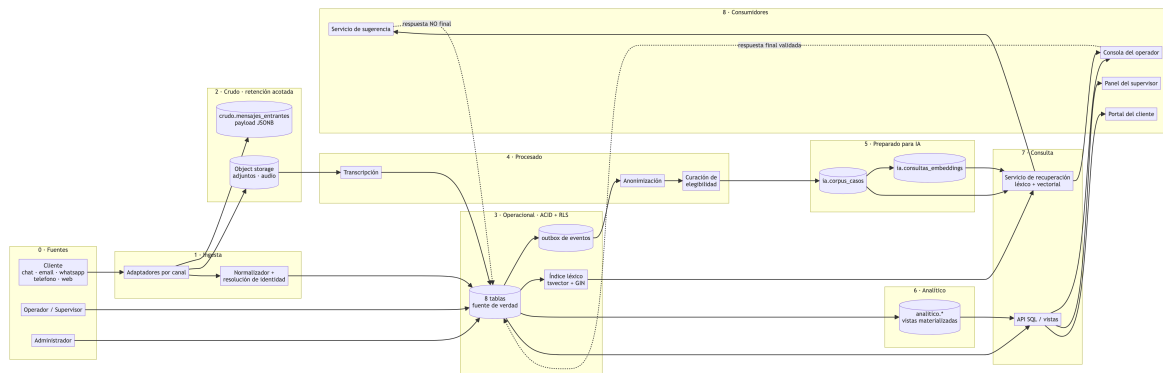


Diagrama de arquitectura de datos (punto 12.2)

El dato entra por un canal, se guarda crudo para poder reprocesarlo, se normaliza hacia el núcleo transaccional, y de ahí salen tres derivados independientes —índice léxico, corpus anonimizado con sus vectores, y vistas de indicadores— que alimentan a los cuatro consumidores. Las flechas punteadas son el **lazo de retroalimentación**: la respuesta que el operador valida vuelve al núcleo y desde ahí al corpus. Es lo que el enunciado llama “conservar el historial para mejorar la atención futura”, y la única parte cíclica del diseño.

12.3 Flujo y componentes

Fuentes. Cinco canales de cliente, más operadores, supervisores, administrador y el sistema de IA. Dato arquitectónicamente decisivo: **los cinco canales convergen en un único esquema**. No son cinco sistemas heterogéneos a integrar, son cinco formatos de entrada del mismo hecho — lo que elimina el principal motivo para construir un Data Warehouse.

Ingesta. Un adaptador por canal (webhook, IMAP, telefonía) que produce un evento uniforme y es el único componente que conoce las particularidades del canal; resolución de identidad; y escritura transaccional al núcleo. Aquí aparece un hueco del modelo: `tickets.id_cliente` es NOT NULL, así que **no se puede abrir un ticket cuyo cliente todavía no se identificó** —lo primero que llega por WhatsApp o teléfono es un número que puede no estar en `clientes`. La salida adoptada es retener el mensaje en la capa cruda hasta poder identificarlo, en lugar de rechazar el contacto o crear clientes provisionales que violarían `UNIQUE NOT NULL` en `dni` y `email`.

Orquestación: tabla `outbox` consumida por un worker. El evento se escribe en la misma transacción que el hecho, así que no puede perderse; es reintentable e idempotente vía `hash_contenido`; y **no agrega infraestructura**. Se descarta `LISTEN/NOTIFY` porque el aviso no es persistente, y una cola externa (Kafka, RabbitMQ) porque a este volumen es un broker más que operar sin beneficio, en la misma línea con que 7.2 descartó Cassandra.

Datos crudos. Payload original en `crudo.mensajes_entrantes` + binarios en almacenamiento de objetos. Se justifica por tres razones concretas: poder **reprocesar la anonimización** cuando el anonimizador mejore (sin el original, cada mejora aplica solo hacia adelante y el corpus viejo queda con fugas permanentes), poder **reprocesar la transcripción** del canal `telefono`, y **auditoría** ante una disputa sobre qué dijo el cliente. No es un Data Lake: es una tabla append-only más un bucket, que nadie consulta analíticamente. Dos decisiones que obliga: **retención acotada** (~90 días) y **el acceso más restringido de todo el sistema**, porque es la única capa con el texto sin anonimizar.

Almacenamiento operacional. Las 8 tablas actuales, sin cambio de naturaleza. Absorbe todas las adiciones del punto 11 (`metadatos_canal`, `metadatos_generacion`, `respuestas.fecha`, `comentario`, derivación explícita en `ticket_logs`, `adjuntos/transcripciones`, `parametros`, `outbox_eventos`) más el índice FTS, que conviene

mantener dentro de esta capa: se calcula desde la misma fila, es transaccionalmente consistente y no requiere anonimizar.

Datos procesados. Transcripción → anonimización → curación de elegibilidad. El orden es la clave: **la capa de IA nunca ve el texto identificable**, y eso es una frontera del flujo, no una política que la aplicación deba recordar aplicar.

Datos preparados para IA. `ia.corpus_casos` (anonimizado y curado) y `ia.consultas_embeddings` (vector + metadatos corregidos en 11.3), en un esquema propio y desactivable.

Almacenamiento analítico. Esquema `analitico` con **vistas materializadas** refrescadas por planificador, no un motor separado. Se materializa porque la consulta de tiempo promedio de resolución agrupa `ticket_logs` completa con dos `FILTER` y un `JOIN` a una vista que resuelve un `LATERAL` por ticket: su costo crece con el histórico, no con el volumen del día. Y `ticket_logs` es la tabla de mayor crecimiento del modelo (~3 filas por ticket, más una por cada derivación y reapertura). Un supervisor mirando un tablero no necesita el dato al segundo: **aceptar consistencia eventual aquí es gratis**, la misma decisión que 6.3 ya tomó para el índice vectorial. Excepción: “temas más consultados” no se puede materializar con SQL porque requiere agrupar por significado — vive en la capa de IA y es la única dependencia real del tablero respecto del componente vectorial.

Componentes de consulta. Vistas y API SQL sobre el operacional (`vw_estado_actual_tickets` ya cumple ese rol); búsqueda léxica FTS; consultas analíticas; y el **servicio de recuperación**, que genera candidatos combinando FTS y ANN, los fusiona, filtra duro por permisos/estado/vigencia/recencia, re-rankea blando por canal/origen/calificación, aplica umbral y —si nada lo supera— **no sugiere y deriva a un humano**. Dos decisiones: la recuperación es **híbrida y no puramente vectorial** (el FTS acierta donde el vocabulario coincide, el vector donde hay sinonimia, y el servicio funciona en modo solo-léxico si la capa vectorial no existe todavía); y el umbral vive en el servicio, no en el consumidor, porque delegarlo garantiza que alguna aplicación termine mostrando el mejor de los malos resultados.

Consumidores y accesos. Portal del cliente, consola del operador, panel del supervisor, consola de administración, servicio de sugerencia, worker de procesamiento y planificador analítico — cada uno con su rol de PostgreSQL y su alcance de RLS. El renglón crítico: arquitectónicamente el servicio de IA **debe consultar con la identidad del usuario que originó la consulta**, no con una identidad de servicio con acceso total — si consulta como servicio, la RLS del operacional queda intacta y perfectamente inútil, porque el contenido sale por el costado semántico. Es la “puerta lateral” del punto 1 expresada como requisito de arquitectura, y es exactamente el criterio que sigue `rol_sistema_ia` (`db/fisico/05_seguridad_permisos.sql`, sección 13): solo inserta sugerencias, sin lectura propia amplia.

12.4 Consistencia y reconstruibilidad

Solo dos capas son autoritativas (crudo y operacional) y **solo una es irremplazable** (operacional): todo lo demás se regenera con un script, así que un respaldo del núcleo más el código de los workers reconstruye el 100% del sistema aguas abajo. La consistencia eventual queda **confinada a lo derivado**: ningún dato que alimente una decisión operativa (estado del ticket, quién lo atiende, calificación) es eventualmente consistente. Todo proceso del flujo debe ser idempotente (`hash_contenido`, `UNIQUE (id_consulta)`) y **reconciliable** por un job periódico que compare el corpus contra el estado transaccional: los triggers fallan y los workers se caen, y una arquitectura que solo confía en el evento acumula deriva silenciosa.

12.5 Justificación del enfoque

Enfoque	¿Aplica?
Simple monolítica	Insuficiente. No tiene dónde retener un mensaje sin cliente identificado, ni dónde anonimizar antes de vectorizar, ni cómo evitar recalcular los indicadores más costosos sobre el histórico en cada tablero
Por capas en un motor	Adoptada

Enfoque	¿Aplica?
Data Warehouse	No. Resuelve integrar fuentes heterogéneas (hay una), aislar cargas analíticas que degraden el OLTP (no a este volumen, y la respuesta barata sería una réplica de lectura) e historizar cambios (ya resuelto por <code>ticket_logs</code>). Costo real: una copia de los datos personales, un ETL que mantener y un segundo modelo de permisos donde el aislamiento por cliente puede perderse sin que nadie lo note
Data Lake	No. Los únicos datos no tabulares del caso son adjuntos y audio, que hoy no existen en el modelo y cuando existan serán binarios con puntero. Sería un bucket vacío con un catálogo que nadie consulta — y en este dominio, el lugar donde los datos personales se acumulan sin dueño ni retención: R1 amplificado
Lakehouse	No. Resuelve un problema derivado de tener un lake
Motor vectorial dedicado	No a esta escala , coherente con 7.3. Se mantiene como opción de evolución

Arquitectura por capas lógicas dentro de un único PostgreSQL, implementadas como esquemas — `public` (fuente de verdad, existe hoy), `ia`, `analitico` — más un bucket de objetos para binarios cuando esas fuentes se incorporen. El argumento decisivo es el mismo con que 7.2 eligió pgvector sobre una base vectorial dedicada: **un solo modelo de permisos**. Cada motor adicional es un lugar más donde el aislamiento por cliente puede estar mal configurado, y en un sistema que guarda DNI y direcciones ese es el riesgo dominante, no el rendimiento. A eso se suma que las transiciones entre capas pueden ser transaccionales (el outbox), que `REV0KE` sobre un esquema da el aislamiento en una línea de DDL, y que se puede construir por partes — que dado que hoy solo existe `public`, no es una ventaja teórica sino la única forma de que el plan sea ejecutable.

12.6 Etapas de implementación

Etapas	Contenido	Habilita
1	<code>respuestas.fecha</code> ; <code>parametros</code> (roles de PostgreSQL y RLS sobre las 8 tablas del núcleo ya implementados y probados, <code>db/fisico/05_seguridad_permisos.sql</code> , sección 13)	Todo lo demás. Falta la fecha para tener recencia
2	Esquema <code>analitico</code> + vistas materializadas + planificador	Tableros sin recalcular sobre el histórico
3	FTS; esquema <code>crudo</code> ; adaptadores; outbox + worker; auto-cierre de tickets	Ingesta real, búsqueda léxica y el reloj que hace crecer el corpus
4	Anonimización; curación; <code>ia.corpus_casos</code> ; <code>pgvector</code> + embeddings; servicio de recuperación híbrido	Sugerencia de respuestas y temas frecuentes

La etapa 4 da la funcionalidad más vistosa del enunciado y es deliberadamente la **última**, porque depende de un corpus que hoy tiene 2 casos elegibles y de un proceso de cierre inexistente.

12.7 Conclusión del punto

El caso requiere una arquitectura por capas: no una arquitectura simple, y tampoco un Data Warehouse, un Data Lake ni un Lakehouse. No simple, porque hay cuatro necesidades que un esquema plano no cubre: retener un mensaje cuyo cliente aún no se identificó sin violar `NOT NULL` ; anonimizar antes de vectorizar como frontera del flujo; dejar de recalcular los indicadores costosos en cada tablero; y separar con permisos distintos el contenido

identificable del apto para IA. Y no los tres enfoques de gran escala, porque el caso tiene una sola fuente de verdad, volumen bajo a mediano, historización ya resuelta por `ticket_logs` y datos casi enteramente tabulares: resolverían problemas que no se presentan y multiplicarían los lugares donde viven los datos personales, que es el riesgo dominante del sistema.

Dos hallazgos que el análisis del flujo agrega y el modelo estático no muestra:

- 1. **Con el criterio de indexación corregido en 11.3, el corpus elegible hoy es de 2 casos de 10 tickets** (los tickets 2 y 10) —y de 1 si se exige validación humana, porque el del ticket 2 tiene respuesta final de IA. La causa: **solo 3 tickets llegan a cerrado** y uno de ellos (el 7) está dado de baja, mientras **5 quedan detenidos en resuelto** sin que exista proceso alguno que los cierre. Falta una regla de **auto-cierre**, que es a la vez una regla de negocio ausente y el reloj que hace crecer la capa de IA. Sin ella, esa capa nunca se llena.
- 2. **“Respuesta final” no implica “reutilizable”**. La respuesta final del ticket 4 —*“El pedido sera entregado durante el dia de hoy”* (`id_respuesta` 6)— es correcta, humana, final y de un ticket activo, y sugerirla la semana que viene es directamente falso. Ningún filtro del punto 11 la detecta: no está desactualizada, no es de IA, no es intermedia. Es **específica del caso**, no conocimiento del dominio. Eso obliga a que la capa de procesado tenga un criterio de generalizabilidad, apoyado en una señal que hoy no existe (`respuestas.reutilizable`, marcada por el operador).

Consecuencia de ambos: la capa de IA debe quedar **diseñada, implementable y desactivada**, y toda la arquitectura tiene que funcionar sin ella —el operador atiende, los indicadores se calculan, la búsqueda léxica responde. Esa propiedad es la que permite construir el diseño en cuatro etapas, empezando por **las identidades de base de datos y la RLS**, citadas como mitigación central desde el punto 2 y ya implementadas y probadas contra Postgres real en `db/fisico/05_seguridad_permisos.sql` (sección 13); lo que queda pendiente de esa primera etapa es `respuestas.fecha` y `parametros`.

13. Estrategia de seguridad, permisos y aislamiento

- **Tipos de usuarios y roles.** El dominio no es multi-tenant (una sola empresa), por lo que el aislamiento relevante es entre clientes entre sí y entre roles internos. Se identifican los mismos cinco actores ya descritos en el punto 1 del informe — cuatro humanos más el propio sistema de IA como actor con permisos diferenciados:
 - **Cliente:** no es un empleado (no tiene fila en `roles`); se identifica por su propio registro en `clientes`.
 - **Operador, Supervisor, Administrador:** son los tres valores de `roles.descripcion` para empleados (`empleados.id_rol`).
 - **Sistema de IA:** actor sin fila propia en `empleados`; opera dentro de la sesión del usuario humano que dispara la consulta (mismas restricciones que ese usuario) y, para sugerir respuestas, solo lee de `consultas_embeddings` — tabla que por diseño excluye consultas no resueltas o respuestas no marcadas como `es_respuesta_final` (`vectorial/modelo_vectorial.md`, punto 1). Esto es la mitigación directa al riesgo **R2** ya identificado en el punto 2 del informe (“fuga de datos sensibles por vía semántica”) y a la advertencia del punto 1 sobre la IA como “puerta lateral” que podría saltar el control de acceso relacional.
- **Control de accesos — matriz de permisos por tabla:**

Tabla	Cliente	Operador	Supervisor	Administrador
<code>clientes</code>	Lee/actualiza su propio registro (<code>id_cliente</code>)	Lee datos de contacto del cliente del ticket asignado (nombre, apellido, email, teléfono principal); no ve <code>dni</code> ni <code>direccion</code> (ver más abajo)	Lee todos	CRUD completo; único rol que puede setear <code>activo = false</code>

Tabla	Cliente	Operador	Supervisor	Administrador
tickets	Crea los propios; lee solo donde <code>id_cliente</code> es el suyo	Lee/actualiza donde <code>id_empleado</code> es el suyo	Lee y reasigna todos	CRUD completo
ticket_logs	Lee el historial de sus propios tickets (solo lectura, vía join a <code>tickets</code>)	Inserta una fila al cambiar el estado de un ticket propio; nunca actualiza ni borra (la tabla es append-only por diseño)	Lee todo	Lee todo; no se habilita <code>UPDATE / DELETE</code> para nadie, ni siquiera admin, para no romper la auditoría
conversaciones, consultas	Crea consultas, califica la conversación al cierre (<code>calificacion</code>) y lee sus propias conversaciones (vía <code>tickets.id_cliente</code>)	Lee/responde las de sus tickets asignados	Lee todas (uso analítico)	CRUD completo
respuestas	Lee las respuestas de sus consultas	Inserta/edita respuestas humanas (<code>es_humano = true</code>) de sus tickets; lee las sugerencias de IA	Lee todas	CRUD completo
empleados, roles	Sin acceso	Lee su propio registro	Lee toda la nómina (simplificación: el esquema físico no modela “equipo” — no hay relación supervisor↔operador, solo <code>id_rol</code>)	CRUD completo
consultas_embeddings	Sin acceso directo	Lectura acotada a los embeddings de sus tickets, filtrando por columnas propias <code>id_cliente / id_ticket</code> (detalle más abajo)	Lectura amplia (analítico)	CRUD; reindexación

- **Datos sensibles identificados** (complementa el relevamiento de riesgos del punto 2): PII de `clientes` — `dni`, `email`, `telefono_1`, `telefono_2`, `direccion`; el texto libre de `consultas.pregunta` y `respuestas.texto_respuesta`, que puede contener datos personales pegados por el cliente (riesgo **R2**); `conversaciones.calificacion` combinada con el resto del caso, que perfila el desempeño de un operador puntual (riesgo **R8**).
- **Mecanismos de aislamiento — implementados en `db/fisico/05_seguridad_permisos.sql`**. Se combinan tres mecanismos: roles nativos de PostgreSQL, Row Level Security (aislamiento por fila) y `GRANT` a nivel de columna (ocultar PII completa a quien no la necesita). Los índices que ya existen en `db/fisico/02_indices_vistas.sql` (`idx_tickets_id_cliente`, `idx_tickets_empleado_activos`) hacen baratas estas políticas, porque filtran por las mismas columnas. Extracto representativo (el script completo cubre las 8 tablas del núcleo):

```

-- Roles NOLOGIN: el backend hace SET ROLE al que corresponda tras
-- autenticar al usuario, dentro de la misma transacción del pedido.
CREATE ROLE rol_app_cliente NOLOGIN;
CREATE ROLE rol_operador NOLOGIN;
CREATE ROLE rol_supervisor NOLOGIN;
CREATE ROLE rol_administrador NOLOGIN;

ALTER TABLE tickets ENABLE ROW LEVEL SECURITY;

-- current_setting(..., true) – segundo argumento missing_ok – para que
-- una sesión sin autenticar DENIEGUE (0 filas) en vez de romper con
-- "unrecognized configuration parameter". Verificado empíricamente:
-- sin el "true" esto lanza error en una conexión que nunca hizo SET.
CREATE POLICY cliente_ve_sus_tickets ON tickets
  FOR SELECT
  TO rol_app_cliente
  USING (id_cliente = current_setting('app.current_cliente_id', true)::int);

CREATE POLICY operador_ve_asignados ON tickets
  FOR SELECT
  TO rol_operador
  USING (id_empleado = current_setting('app.current_empleado_id', true)::int);

-- Una vez habilitado RLS, cualquier rol SIN policy propia ve 0 filas
-- (no "todo"): supervisor y administrador necesitan su propia policy
-- explícita para ver todo, no alcanza con "no restringirlos".
CREATE POLICY supervisor_admin_ven_todo_tickets ON tickets
  FOR SELECT
  TO rol_supervisor, rol_administrador
  USING (true);

-- El mismo criterio se propaga a conversaciones/consultas/respuestas
-- vía subquery sobre tickets, para no duplicar la condición de
-- aislamiento (ver script completo).
--
-- consultas_embeddings es la única excepción a ese patrón de subquery:
-- el motivo de desnormalizar esa tabla (6.3b) es evitar un JOIN contra
-- el núcleo en el camino caliente de la búsqueda por similitud, así
-- que su policy no puede reintroducirlo. Por eso lleva columnas
-- propias id_cliente/id_ticket (duplicadas, igual que
-- contenido_pregunta/contenido_respuesta), y su policy es una
-- comparación de igualdad indexada, no un JOIN:
--   USING (id_cliente = current_setting('app.current_cliente_id', true)::int)
-- Pendiente hasta que la tabla se cree físicamente (no existe hoy).

-- Columnas de PII de clientes ocultas para el operador: nombre,
-- apellido, email y teléfono principal alcanzan para personalizar y
-- responder por cualquier canal; dni, dirección y teléfono
-- alternativo no son necesarios para resolver un caso.
GRANT SELECT (id_cliente, nombre, apellido, email, telefono_1)
  ON clientes TO rol_operador;

```

Dos correcciones que solo aparecieron al ejecutarlo contra un Postgres real (no se ven leyendo el SQL): **(1)** `security_invoker = true` en `vw_estado_actual_tickets` no alcanza solo — también hace falta `GRANT SELECT` sobre la vista como objeto, o Postgres devuelve `permission denied for view` aunque el rol tenga acceso a las tablas base; **(2)** un `RESET` de la variable de sesión no la vuelve a dejar en “nunca seteada” (queda en `''`, que rompe el cast a `int`) — para producción, la recomendación es usar `SET LOCAL` (no `SET`) dentro de la transacción de cada pedido, para que revierta sola al hacer `COMMIT`/`ROLLBACK` sin depender de que el backend se acuerde de resetearla. Sin esto, una conexión reciclada por un pool podría heredar el `id_empleado` de un pedido anterior — el riesgo real que esta sección busca evitar. Detalle completo en los comentarios de `db/fisico/05_seguridad_permisos.sql`.

- **Mitigación de riesgos de exposición indebida vía IA.** Además de que el “usuario” IA opera con los permisos de quien pregunta (punto anterior), `consultas_embeddings` solo indexa **casos resueltos y validados** (nunca respuestas intermedias o descartadas), lo que ya limita qué puede sugerirse. Falta, y se

deja como pendiente de implementación, aplicar RLS sobre `consultas_embeddings` mediante sus propias columnas `id_cliente / id_ticket` (ver más arriba), tal como pide `vectorial/modelo_vectorial.md` en su punto 5 (“debe respetar la misma política de aislamiento... un operador o un cliente no deben poder recuperar, vía búsqueda semántica, el contenido de un caso al que no tendrían acceso por la vía relacional normal”). El filtro debe aplicarse **antes** de la búsqueda por similitud (a nivel `WHERE` de Postgres, no post-procesado en la aplicación), para que una fila fuera de alcance ni siquiera participe del `ORDER BY embedding <-> ...`.

- **Auditoría.** `ticket_logs` ya es, por diseño, un registro append-only con `fecha` e `id_empleado` responsable (`db/logico/restricciones.md`), y ninguna tabla del núcleo admite `DELETE` físico (soft delete vía `activo`, mitigación ya documentada para el riesgo **R5**). Se propone extender el mismo criterio de “nunca sobrescribir, siempre agregar” a los accesos de lectura sobre datos sensibles de `clientes` que ocurran fuera del flujo normal de atención de un ticket (por ejemplo, una consulta de un supervisor sobre un cliente puntual sin ticket activo asociado), y reforzar que `respuestas.es_humano / es_respuesta_final` ya cubren la trazabilidad IA-vs-humano que pide la consigna.
- **Implementado y probado — `db/fisico/05_seguridad_permisos.sql`.** El script es aditivo: no modifica `01_creacion_tablas.sql`, `02_indices_vistas.sql` ni `04_validaciones.sql` (la corrección de `security_invoker` sobre la vista es un `ALTER VIEW` dentro del script nuevo, no una edición del archivo de Fran). Se ejecutó de punta a punta contra una instancia real de PostgreSQL 18 — `01` → `02` → `03` (datos de ejemplo) → `05` — y se verificaron con `SET ROLE` + variables de sesión, entre otros: un operador solo ve sus 3 tickets asignados (vía tabla y vía la vista); un cliente solo ve sus propios tickets; supervisor ve los 10; una sesión sin autenticar ve 0 filas en lugar de romper; el operador no puede leer `clientes.dni`; nadie puede `UPDATE / DELETE` sobre `ticket_logs`; el rol de servicio de IA puede insertar sugerencias (`es_humano = false`) pero no puede insertar haciéndose pasar por humano, ni el operador puede responder consultas de tickets ajenos. Las 13 validaciones de `04_validaciones.sql` se corrieron después y siguen pasando sin cambios. No quedó nada corriendo: la instancia de prueba se creó y se destruyó por completo, no se tocó ningún servicio del entorno del usuario.

14. Consideraciones de escalabilidad y rendimiento

- **Puntos críticos de crecimiento.**

Estructura	Por qué crece	Proporción de partida (datos de ejemplo)	Qué se rompe primero al escalar
<code>ticket_logs</code>	Append-only por diseño (nunca <code>UPDATE / DELETE</code> , sección 13): cada cambio de estado de cada ticket agrega una fila, para siempre	31 filas / 10 tickets ≈ 3,1 por ticket	Es la única tabla del núcleo cuyo crecimiento no está acotado por cantidad de entidades, sino por entidades × eventos × tiempo. Con miles de tickets/mes sostenidos durante varios años sería la primera en llegar a varios millones de filas. No se rompe la lectura de un ticket puntual (el índice compuesto la mantiene barata a cualquier volumen) — se rompen las operaciones sobre la tabla completa: <code>VACUUM / ANALYZE</code> , backups, o un indicador histórico sin filtro de fecha

Estructura	Por qué crece	Proporción de partida (datos de ejemplo)	Qué se rompe primero al escalar
conversaciones / consultas / respuestas	Crece con el volumen de interacción, acotadas por ticket pero no por el tiempo que ese ticket permanece abierto	10 / 12 / 17 filas (~1,4 respuestas por consulta)	La Consulta 3 (GROUP BY es_humano sin ventana de tiempo sobre toda respuestas): un agregado sin filtro de fecha escala linealmente con el total histórico, no con la actividad reciente que un tablero de gestión realmente necesita
clientes / empleados / tickets	Acotadas por cantidad de entidades, pero nunca se achican : la baja es lógica (activo = false , sección 13), no DELETE físico	Volumen bajo hoy	Crece más despacio que ticket_logs (sin el eje tiempo), pero un índice sobre estas tablas carga indefinidamente filas ya inactivas para el negocio — no hay purga que lo compense
consultas_embeddings (si se materializa)	Solo indexaría casos cerrados y con respuesta final validada (vectorial/modelo_vectorial.md , punto 1)	Corpus elegible hoy: 2 casos sobre 10 tickets (punto 11)	Si la funcionalidad de sugerencia de respuestas tiene éxito, este corpus crece con el mismo eje temporal que ticket_logs — y es ese crecimiento, no una fecha en el calendario, el que convierte al scan secuencial (hoy más exacto que HNSW , ver más abajo) en el cuello de botella

- **Estrategia de optimización.** Frente a cada punto crítico de la tabla anterior, la solución ya está en uno de dos estados: **construida**, o **decidida con un disparador concreto para construirla**. En ningún caso queda para pensar en el momento.

Ya construido — decisiones tomadas antes de que hubiera un problema de escala real:

- **Índices para lectura acotada (db/fisico/02_indices_vistas.sql).** Un índice por cada FK del camino caliente (idx_tickets_id_cliente , idx_tickets_empleado_activos , idx_ticket_logs_ticket_fecha , idx_conversaciones_id_ticket , idx_consultas_id_conversacion , idx_respuestas_id_consulta) ya garantiza que leer “los tickets de un cliente” o “el último estado de un ticket” sea una búsqueda logarítmica, no un recorrido completo, a cualquier volumen. La sección 13 ya señala que esto abarata las políticas RLS sobre tickets ; vale extender la observación a conversaciones / consultas / respuestas , donde la policy filtra vía una subconsulta que hace JOIN hasta tickets (la de consultas , por ejemplo, recorre conversaciones INNER JOIN tickets) — esos mismos índices de navegación son los que sostienen esa cadena a escala, sin duplicar índices para seguridad y para performance por separado.
- **tickets.id_empleado desnormalizado (punto 6.3a).** No es una optimización pendiente: ya está en el esquema implementado, y se justificó específicamente para que “qué tickets tiene asignados un operador ahora” —la consulta más frecuente del sistema— nunca dependa de recalcular sobre ticket_logs , que es la tabla que más crece. Es preparación para escala decidida antes de que existiera un problema de escala.
- **Roles nativos + Row Level Security (db/fisico/05_seguridad_permisos.sql , sección 13).** El aislamiento por cliente/operador ya corre a nivel de motor, ejecutado y probado contra Postgres real. Cuando crezca la cantidad de usuarios no hay rediseño pendiente: es la misma implementación de hoy operando sobre más filas y más roles activos a la vez.

Ya decidido — sin construir todavía, pero con umbral y mecanismo ya especificados, no a definir cuando el volumen aparezca:

- **La separación del componente vectorial (punto 6.3b).** La decisión de diseño ya está tomada, y justificada específicamente por escala —que la búsqueda semántica no compita por I/O con la atención de tickets en curso—, aunque `consultas_embeddings` no exista todavía como tabla física (así lo dejan explícito los puntos 11, 12 y 13). Cuando se cree, nace ya separada; no es algo a decidir en el momento en que el volumen lo exija.
- **Dos índices con condición de activación ya definida:** si el planner no puede usar el único índice existente sobre `tickets.id_empleado` (`idx_tickets_empleado_activos`, parcial, `WHERE activo = TRUE`) para la policy `operador_ve_asignados` —que filtra sin esa condición— (conviene verificarlo con `EXPLAIN`), el reemplazo ya está identificado: un índice simple `idx_tickets_id_empleado`. Y un índice de texto completo (`tsvector` + `GIN`, `spanish` + `unaccent`) sobre `consultas.pregunta / respuestas.texto_respuesta` ya está propuesto por el punto 12 (etapa 3) para el momento en que exista búsqueda léxica por palabra clave, hoy inexistente.
- **Particionamiento de `ticket_logs` por rango de fecha**, para cuando las operaciones sobre la tabla completa (`VACUUM`, `ANALYZE`, backups, un indicador histórico sin filtro) empiecen a degradarse pese a los índices puntuales — es la única tabla que crece también en el eje tiempo (fila anterior). El mecanismo ya está especificado: particionamiento declarativo de PostgreSQL (`PARTITION BY RANGE` (fecha), particiones mensuales o trimestrales), compatible con “nunca borrado físico” porque particionar no es eliminar.
- **Vista materializada para las consultas agregadas.** Las Consultas 3 y 5 (punto 10) recalculan sobre el histórico completo en cada ejecución, sin ventana de tiempo — el primer patrón en volverse costoso a medida que crece el total. El reemplazo ya está escrito, en el esquema `analitico` que propone el punto 12 (etapa 2), listo para crearse cuando el refresco periódico deje de ser prematuro:

```
-- Propuesta, no implementada -- vive en el futuro esquema `analitico`
-- (punto 12, etapa 2), no en db/fisico/.
CREATE MATERIALIZED VIEW analitico.tickets_por_canal_y_estado AS
SELECT canal_origen, estado_actual, COUNT(*) AS cantidad_tickets
FROM vw_estado_actual_tickets
WHERE activo = TRUE
GROUP BY canal_origen, estado_actual;

-- Refresco periodico (planificador externo o pg_cron), no en
-- cada lectura del tablero:
REFRESH MATERIALIZED VIEW CONCURRENTLY analitico.tickets_por_canal_y_estado;
```

- **Separación operacional/analítica**, incluidos los esquemas por capas del punto 12 (`analitico`, `ia`): ninguno existe todavía, pero el punto 12 ya fija tanto el disparador (consultas agregadas compitiendo por recursos con la operación) como la acción (réplica de lectura, no un Data Warehouse aparte — esa complejidad ya se descarta a esta escala en la sección 7.3).
- **HNSW sobre `consultas_embeddings`**, para cuando el corpus (que crece con el mismo eje temporal que `ticket_logs`, si la funcionalidad de sugerencia tiene éxito) deje de caber en un scan secuencial exacto. Hoy, con un corpus de dos dígitos, el scan secuencial no es solo suficiente: es *más exacto* que HNSW, que es un índice aproximado (punto 11) — construirlo ahora perdería precisión sin ganar velocidad. El umbral y el mecanismo ya están identificados; también una advertencia para cuando llegue: combinar HNSW con un filtro muy selectivo (un `WHERE id_cliente = ...` de RLS) puede devolver menos de k resultados, así que la estrategia ya decidida es filtrar primero y calcular distancia exacta sobre el subconjunto filtrado, no al revés.
- **Connection pooling para usuarios concurrentes.** Más clientes y operadores conectados a la vez agota conexiones de Postgres antes de que cualquier tabla se vuelva grande. El mecanismo estándar (pgbouncer o similar) ya tiene resuelta la parte que podría romperlo: la sección 13 ya identificó y

verificó que hace falta `SET LOCAL`, no `SET`, para que una conexión reciclada entre pedidos de usuarios distintos no arrastre el `id_cliente / id_employado` de un pedido anterior. Activar pooling no requiere resolver esa corrección en el momento; ya está resuelta.

La frontera entre lo ya construido y lo ya decidido no es arbitraria: la misma pregunta que ya justificó la elección de motor (punto 7) —el compromiso entre **simplicidad, rendimiento, consistencia y costo**— es la que la traza. Lo construido son piezas sin costo de mantenimiento propio ni sacrificio de consistencia: índices, una columna desnormalizada, roles y RLS que ya hacían falta por seguridad de todos modos. Lo decidido-pero-no-construido son piezas que, activadas hoy, agregarían infraestructura para pagar y operar (réplica, motor vectorial) o consistencia para sacrificar (vista materializada con refresco periódico, `HNSW` aproximado) sin que exista todavía volumen que lo justifique — la vista materializada queda desactualizada entre refrescos, la réplica tiene *lag*, el índice vectorial ya acepta consistencia eventual por diseño (punto 6.3b); el núcleo transaccional no cede nunca esa consistencia. El punto 12 ya fija el umbral medible de cada una: la arquitectura no necesita improvisar cuando ese umbral llegue, solo ejecutar lo que ya está escrito.

15. Conclusiones

- **Balance del diseño.** La solución adoptada es un núcleo relacional normalizado hasta 3FN sobre un único motor (PostgreSQL + pgvector), con una extensión vectorial acotada a dos columnas y una capa de seguridad implementada a nivel de motor, no de aplicación (puntos 6, 7 y 13).

Dos hallazgos no eran visibles en el modelo estático y solo aparecieron al trazar el flujo real de los datos de ejemplo (punto 12.7). El primero: con el criterio de indexación correcto, el corpus de casos elegibles para el componente de IA es hoy de **2 sobre 10 tickets** (los tickets 2 y 10), porque 5 de los 10 quedan detenidos en `resuelto` sin que exista ningún proceso que los cierre — falta una regla de auto-cierre. Construirlo requiere infraestructura que no está en el alcance: `ticket_logs.id_employado` es `NOT NULL`, así que un cierre automático necesita un actor a quien atribuírselo, y el proyecto no tiene ningún mecanismo de ejecución periódica corriendo. Esa infraestructura sí está definida: el punto 12.6 la ubica en la etapa 3 de un plan de cuatro, junto con el patrón `outbox + worker` que el punto 12.3 describe en detalle — después de la etapa 1 (identidades y RLS), la única ya construida y probada contra Postgres real.

El segundo hallazgo: “respuesta final” no equivale a “reutilizable”. La respuesta del ticket 4 (“El pedido será entregado durante el día de hoy”) es correcta, humana y final, y sugerirla la semana siguiente sería directamente falsa. Ninguno de los tres filtros que el diseño sí tiene la detecta — no está desactualizada (`vigente` solo cambia si el ticket se reabre, y este no se reabrió), no es de IA (`es_humano = true`), no es intermedia (`es_respuesta_final = true`) —, porque el problema no es ninguno de esos tres: la respuesta es específica de ese momento, no conocimiento general del dominio. Los dos hallazgos comparten origen: aparecen al seguir un caso de punta a punta, no al mirar el diagrama de tablas.

La lección más honesta que deja el proyecto es la asimetría entre lo diseñado y lo construido: de todo el sistema, la única pieza que se implementó y se probó contra una instancia real de PostgreSQL es la capa de seguridad — roles, Row Level Security y `GRANT` (punto 13). El resto (arquitectura por capas, componente vectorial, búsqueda léxica, auto-cierre) quedó en diseño justificado y secuenciado, no en código corriendo. No es una falla: la consigna no exige una implementación avanzada, exige justificación técnica razonable, y esa justificación es más sólida cuando distingue con claridad qué se verificó de qué se argumentó.

El compromiso entre simplicidad, rendimiento, consistencia y costo aparece una y otra vez, y siempre se resolvió del mismo lado: un motor único en vez de un stack políglota (punto 7), una arquitectura por capas dentro de ese mismo motor en vez de un Data Warehouse (punto 12), e índices y una columna desnormalizada ya construidos porque no cuestan mantenimiento extra, frente a réplicas, particionamiento o un índice vectorial dedicado que quedan decididos pero sin activar hasta que un umbral medible lo justifique (punto 14). La única consistencia que el diseño sacrifica es la del componente derivado — el índice semántico, las vistas analíticas, nunca la del núcleo transaccional: el estado de un ticket, quién lo atendió y quién respondió se mantienen con consistencia fuerte en todo momento.

La capa de IA queda, en consecuencia, diseñada, justificada y deliberadamente sin activar (detalle de por qué en el punto 11 y en las limitaciones del README) — el sistema completo funciona sin ella, que es la condición que permitió construirlo por etapas desde el principio.